

CODE SPORT



Sandya Mannarswamy

In this month's column, we continue our discussion on the testing of machine learning applications.

Last month, we had an overview of testing natural language processing applications. Subsequently, I received email requests from a couple of readers to broaden this discussion to include machine learning programs, in general. Hence, in this month's column, we will explore the testing of machine learning applications as well.

An important point to remember is that a machine learning application or model is typically part of a larger software system. For instance, consider a software system deployable on your mobile, with which you can point to an image/photo and the system can translate the text embedded in the image to any language of your choice, in real-time. This software system consists of a number of components, many of which are traditional software components and a couple of machine learning components.

The software includes one machine learning (ML) model, which extracts the text embedded in the image uploaded, and a second ML model that can recognise the source language of the text and machine translate it in the target language specified by the user, in real-time. These two ML components interact with many of the traditional software components that are part of this overall system. Hence, when we talk about testing the ML applications, we need to test the ML components themselves, as well as their interaction with other traditional software components.

When testing ML components, one needs to focus on three major sub-components of the ML application -- data, features and the model/algorithm. Another way of looking at the ML applications is to understand the testing needs for each stage of the app's life cycle.

So what are the stages in the life cycle of an ML application? Broadly, they are: (a) Input data management, (b) Model learning, (c) Model verification, and (d) Model deployment.

The input data management stage typically includes data collection for training and validating the ML model. This includes collecting training, validation and test data sets, data labelling and data augmentation. Each of these activities in the data management stage needs to be tested and their software quality ensured. Ensuring the quality of input data is critical in the case of ML applications since the models learn from the data itself, unlike traditional software applications for which the programmer explicitly specifies the logic of the application.

In ML applications, the weights/parameters of the function/model are learnt directly from the input data. Hence, the model's performance and its ability to extract insights, make predictions, derive values and prescribe action will directly depend on the quality of the input data. The input data must be a good mirror of the data from the real world, where the machine learning model will be deployed. Hence, when performing quality testing of input data, one needs to ensure that all these criteria are taken into account during the data management stage of the ML pipeline.

The second stage in the ML application's life cycle is model learning. This is when the model parameters are learnt from the training data set. This also includes the selection of the ML algorithm and the corresponding hyper-parameters. This stage includes selecting the appropriate ML algorithm based on the problem type and domain. The problem type can be about supervised or unsupervised learning, and about predicting a continuous variable (regression) or a categorical variable (classification).

So far, we have not explicitly covered the feature extraction phase in our discussion. In the case of classical ML systems, this would be part of the model learning stage. For deep learning systems, no explicit feature extraction is needed. The quality